# Email Spam Classification

## Using NLP and Machine Learning

Support Vector Machine · TF-IDF · SpamAssassin Corpus

**Digital University Kerala · Predictive Analytics**
Abikrishnan M S · Mohammed Yazin N · Athulya Mannambath

# Presentation Outline

**01** **Problem Statement**
Why spam detection matters & project goals

**02** **Dataset Overview**
SpamAssassin corpus —4,198 emails, class distribution

**03** **Methodology & Pipeline**
Preprocessing → TF-IDF → Model Training

**04** **Key Results**
Naive Bayes vs SVM · Accuracy · Confusion matrix

**05** **Live Application**
Interactive Streamlit web app screenshots

**06** **Conclusions & Future Work**
Takeaways, limitations, and next steps

# Problem Statement & Motivation

## Phishing & Malware

Spamisaprimaryvectorfor phishing attacks, credential theft, and malware distribution.

## Productivity Loss

Users lose significant time managing unsolicited email clutter every day.

## False Positives Matter

Misclassifyinglegitimateemails as spam erodes user trust —precision is cri tical.

## Project Goal

Buildanaccurate, interpretable spam filter using classical NLP & ML with high p reci si on .

**45%** of global email traffic is spam

# Dataset Overview

Source: SpamAssassin Public Corpus · kaggle.com/datasets/beatoa/spamassassin-public-corpus

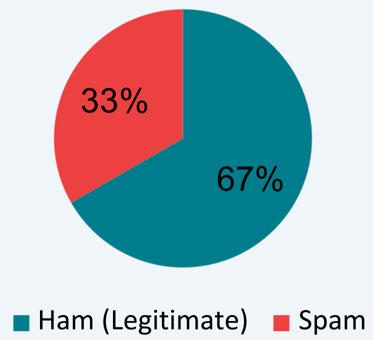
**4,198**
Total Emails

**2,801**
Ham (66.7%)

**1,397**
Spam(33.3%)

**80/20**
Train/TestSplit

| Feature | Configuration |
|---|---|
| Vocabulary Size | 5,000 TF-IDF features |
| N-gram Range | Unigrams + Bigrams (1,2) |
| TF Scaling | Sublinear TF scaling enabled |
| Training set | 3,358 emails |
| Test set | 840 emails |



# Methodology & NLP Pipeline

1. **Raw Email Input** — Headers + Body
2. **Header Stripping** — Isolate body text
3. **Text Normalisation** — Lowercase · URLs → token
4. **Stopword Removal** — NLTK English list
5. **Lemmatisation** — WordNet Lemmatizer
6. **TF-IDF Vectorisation** — 5k features (1,2)-gram
7. **SVM Classifier** — LinearSVC + Platt scaling

## Two Classifiers Compared

### Multinomial Naive Bayes

**Accuracy:93.7% · F1:90.8%**

False Positives: 33 · False Negatives: 20

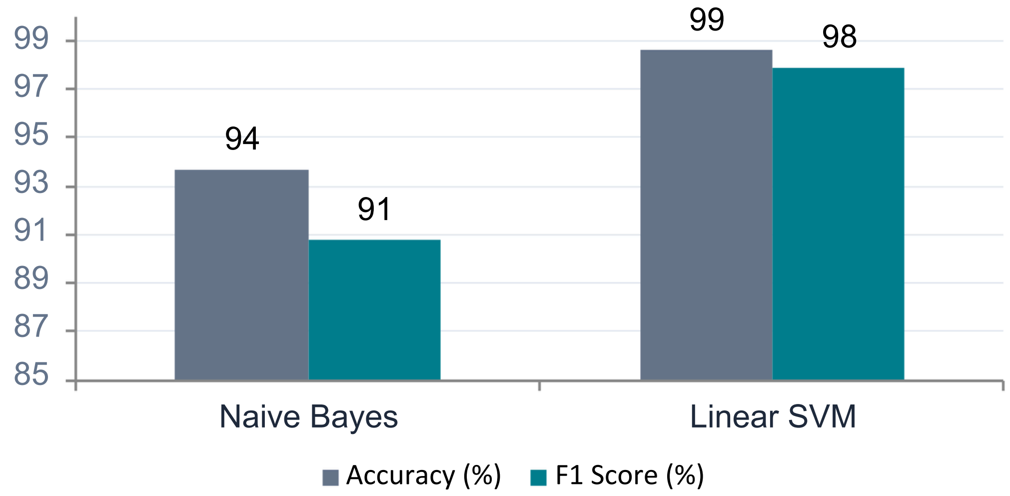
### Linear SVM (LinearSVC)

**Accuracy:98.6% · F1:97.9%**

False Positives: 10 · False Negatives: 2

# Key Results & Model Comparison

## Accuracy & F1 Score Comparison




## Confusion Matrix —SVM

| ActualHam → | ActualSpam → |
|---|---|
| 550 ✓ True Ham | 10 ✗ Fa lse S pam |
| 2 ✗ False Ham | 278 ✓ True Spam |

**98. 6%**
Test Accuracy

**97. 9%**
F1 Score

**1.8%**
False Positive Rate

**0.7%**
FalseNegative Rate

# Model Performance —Detailed Metrics

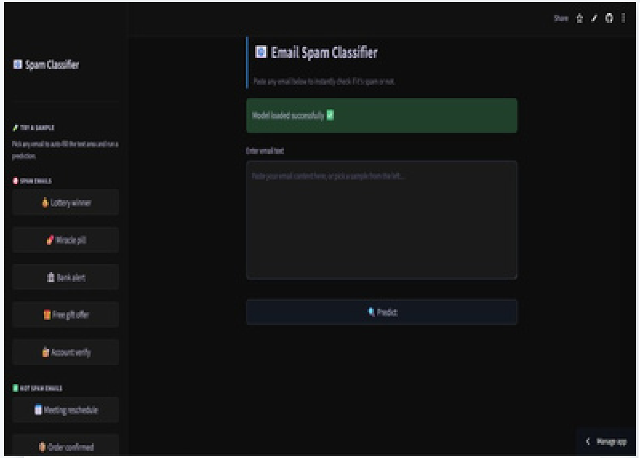
| Me tric | Naive Bayes | Linear SVM ⭐ |
|---|---|---|
| Accuracy | 93.7% | **98.6%** |
| F1 Score | 90.8% | **97.9%** |
| False Positives | 33 | **10** |
| False Negatives | 20 | **2** |
| Probability Output | Nat iv e | Platt Scaling (C alibrate dCla s sif ierC V ) |

## Why SVM Outperforms

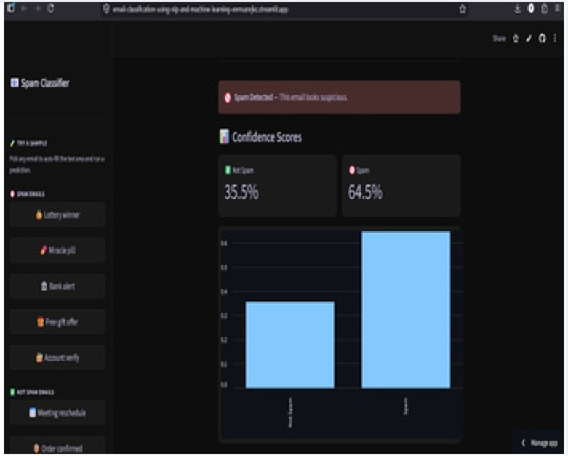
- SVM maximises the margin between classes, making it more robust to overlapping feature distributions.
- TF-IDFsparsehigh-dimensionalspacessuit linear SVMs —they converge efficiently with Li ne arSV C .
- Naive Bayes assumes feature independence, which is violated by bigram features.
- SVM's 10 false positives vs NB's 33 —70% reduction, crucial for user trust.
- Platt scaling provides calibrated probabilities for confidence scoring in the web app.

LinearSVC + TF-IDF (10k features, 1-2 grams) · Trained on 3,358 emails · Evaluated on 840 held-out emails

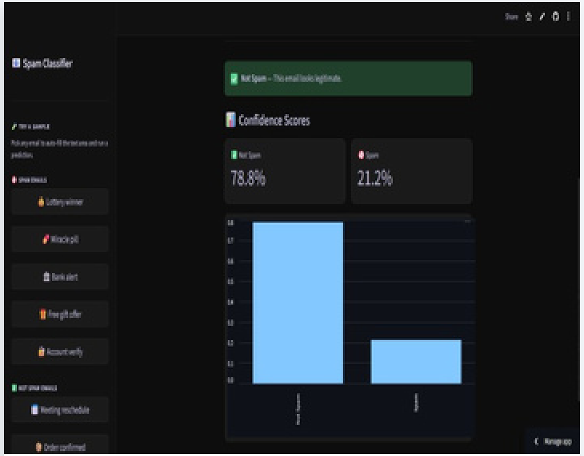
# Deployed Streamlit Web Application




Home —Input Interface




Spam Detection Result




Ham Detection Result

## Application Features

- Paste rawemailtext (including headers) for instant classification
- Visual confidence bar showing ham/spam probability score
- One-clicksampleemails forquickdemo testing
- Expandable preprocessed text view for model transparency

LiveDemo:https://email-classification-using-nlp-and-machine-learning-enmusnrjkc.streamlit.app/

# Conclusions & Limitations

## ✅ Key Conclusions

1. Linear SVM + TF-IDF achieves 98.6% accuracy on the SpamAssassin corpus
2. 70% fewer false positives vs Naive Bayes — critical for user trust
3. Classical NLP pipeline is highly effective for email spam detection
4. Platt scaling enables calibrated confidence scores for UI display
5. End-to-end deployment demonstrates production readiness of the model

## ⚠️ Limitations

- Static model — requires periodic retraining as spam evolves
- English-only — no multilingual or Unicode spam handling
- No adversarial robustness — obfuscated spam may bypass detection
- Dataset skew: 66.7% ham may bias towards reducing false positives
- No image or attachment-based spam detection (text-only pipeline)

# Future Work & Enhancements

### Transformer-Based Models

Fine-tune BERT or DistilBERT for context-aware spam detection and improved accuracy on subtle phishing emails.

### Multilingual Support

Extend the pipeline to handle non-English emails using multilingual embeddings and language detection.

### Online / Active Learning

Implement incremental learning so the model adapts continuously as new spam patterns emerge in real-time.

### Multi-Modal Detection

Analyse embedded images, attachments, andHTML structurefor richerfeature extraction beyond plain text.

### Adversarial Robustness

Test against obfuscated spam (character substitution, homoglyphs) and train with adversarial augmentation.

### Explainability (LIME/SHAP)

Integrate LIME or SHAP to generate per-emailfeatureimportanceexplanationsin the web app.

# Thank You

Email SpamClassificationUsingNLPandMachineLearning

Abikrishnan M S · Mohammed Yazin N · Athulya Mannambath
Digital University Kerala · Predictive Analytics

🔗 github.com · Live Demo: streamlit.app